

Final Coursework Report

Camfranglais

Lexical and Syntactic Analysis of Informal Urban Communication in Yaoundé

STUDY OVERVIEW

We study twelve unchanged transcriptions with our regex lexer and corpus-derived LL(1) parser. We present the original wording, lexical results, slang and topic readings, grammar calculations, automata, complete traces and screenshots of the working application.

AUTHOR GROUP

Name	Matricule
Kwete Ngouba Junior Rayan	ICTU20241377
Djemtchimo Noukui Bruno Jonatan	ICTU20241585
Amina Boubakary	ICTU20241870

Version 4.3 | 28 September 2026

Faculty of Information and Communication Technologies

Instructor: Engr. Tanwi Nkiamboh

Assignment due: 29 September 2026

Abstract and Evidence Statement

We built Camfranglais to make the lexical and syntactic analysis of informal expressions visible. We preserve spelling and word order, split each statement with regular expressions, assign lexical categories and test the resulting category sequence with an LL(1) parser.

We evaluate the twelve transcriptions S01–S12 exactly as stored in Collection. One controlled pass produces 81 tokens and 55 case-folded forms. Our original grammar has ten nonterminals and 37 alternatives. Removing direct left recursion and applying left factoring produce twelve nonterminals, 39 alternatives and 47 populated predictive-table cells, with no conflict. Ten statements parse completely. S09 and S10 reject after five consumed tokens, at the unchanged forms `n'ais` and `alli`.

We also discuss the difference between an emitted software label and a word's role in its sentence. Our linguistic reading identifies 15 slang forms in 17 occurrences, while the unchanged lexer emits zero SLANG labels for this corpus. We relate each statement to its main topic using the words and French gloss recorded with it.

The application opens without an account. Visitors can analyze and save tests, then inspect their original results. We use genuine browser captures from 27 September 2026 to illustrate this workflow.

Controlled evaluation	Measured result
Raw statements / original token occurrences	12 / 81
Known-category occurrences / unresolved occurrences	79 / 2
Emitted language-transition clues	26
Accepted / rejected category sequences	10 / 2
LL(1) conflicts / residual left-recursive nonterminals	0 / 0
Additional constructed boundary controls	7

Reading the results

We use the 83.3% grammar-match rate to describe these twelve category sequences; it does not measure language-wide accuracy.

Contents

Abstract and Evidence Statement	2
1 Context, Problem and Objectives	4
2 Corpus Entry, Preparation and Ethics	5
3 Raw Corpus: Statements S01–S06	6
4 Raw Corpus: Statements S07–S12	7
5 Custom Lexical Specification	8
6 Finite-State Model of the Lexer	9
7 Lexicon, Category Policy and Notation	10
8 Token Tables: S01–S04	11
9 Token Tables: S05–S08	12
10 Token Tables: S09–S12	13
11 Frequency, Coverage and Observed Variation	14
12 Code-Mixing, Phrases and Linguistic Complexity	15
13 Constructing a Corpus-Derived Grammar	16
14 Grammar Transformations	17
15 Transformed Production Catalogue	18
16 FIRST and FOLLOW Sets	19
17 Complete LL(1) Predictive Table	20
18 Pushdown Automaton for the LL(1) Parser	21
19 Complete Accepted Trace: S02	22
20 Complete Rejected Traces: S09 and S10	23
21 Evaluation Results and Boundary Controls	24
22 Working Analyzer: Browser Evidence	25
23 Topic Patterns in the Corpus	26
24 Application Architecture	27
25 Verification, Reproducibility and Limitations	27
26 Conclusion	28
References and Evidence Register	29

1 Context, Problem and Objectives

The assignment and the problem

CS4110 SET A asks a group of three to examine 10–15 real-life expressions heard in Yaoundé, preserving slang, code-mixing, incomplete speech and nonstandard forms. The required compiler work includes token categories and regular expressions, frequency and variation analysis, a grammar constructed from the expressions, grammar transformations, FIRST/FOLLOW, a parsing table and accepted/rejected test cases. Our chosen parsing route is LL(1); an LR automaton is not additionally required for that route.

We face a different problem from compiling a conventional programming language: everyday speech depends on context and varies between speakers. In our corpus, `school` occurs within a French-framed expression. We therefore preserve the wording first and discuss spelling and meaning separately from the computed categories.

Ebongue’s study distinguishes speaker-written Camfranglais from researcher transcriptions and discusses variation in spelling, syntax and morphosyntax [2]. Stored text and computed categories are observable, but a small textual sample cannot establish how a community speaks.

Operational objectives

1. Preserve all twelve original strings and label every token in source order.
2. Apply deterministic recognition rules and expose unknowns, ambiguous labels, language clues and corpus frequencies.
3. Derive reusable category productions from observed structures rather than hard-code a whitelist of twelve complete sentences.
4. Perform grammar transformations and compute every set/table cell.
5. Demonstrate predictive parsing, successful consumption and precise failure positions, including examples outside our twelve-statement corpus.
6. Present an accessible public workspace with durable, immutable test evidence and no unnecessary account workflow.

Scope

We concentrate on lexical analysis, category-level syntax and visible intermediate results. Translation, speech recognition and semantic parsing are outside this study.

2 Corpus Entry, Preparation and Ethics

Entering the statements

We first entered our twelve-statement corpus into the compiler for testing on **22 September 2026**. We credit this corpus-entry work jointly to **Kwete Ngouba Junior Rayan, Djemtchimo Noukui Bruno Jonatan** and **Amina Boubakary**. The entry date refers to our compiler testing session.

We use identifiers S01–S12 throughout the report. Each record keeps the original statement beside its French gloss, so we can compare the wording, its interpretation and the compiler’s output.

Preparation stage	Our record
Initial compiler input	22 September 2026; twelve statements processed in the initial local analysis.
Report analysis snapshot	27 September 2026; the twelve report statements and the grammar used for our final calculations.
Raw text	Capitalization, apostrophes, spelling and word order preserved.
Meanings and topics	French glosses kept separately; topic readings explained through the exact words in each statement.

The initial test entry used `ali` in S10; our report keeps `alli`, as recorded in the 27 September snapshot. We use that snapshot consistently for every token table and parser trace.

Working with informal spelling

We had to separate the written form from our interpretation of it. For example, we kept `sharp` and `shap` as distinct spellings and preserved the contractions in `J’ai` and `n’est`. We also left `n’ais` and `alli` unchanged so that their effect on lexical recognition and parsing remains visible.

Preserving the corpus

We checked every raw statement against its regression fixture. Our 27 September analysis snapshot contains the same twelve Sentence entries and one separate Word annotation, `choa`. We exclude that Word entry from the sentence and token totals.

For each statement, we retain all tokens in source order, calculate frequencies and inspect the parser trace. Section 23 brings these formal results back to everyday themes such as rest, school, bargaining, electricity and transport.

Ethical handling

We refer to statements by their identifiers and omit speaker names. We keep raw wording, meanings and analytical notes separate. The public application warns that submitted test text is visible and retained, so we exclude personal or confidential information from demonstrations.

3 Raw Corpus: Statements S01–S06

We use report-local identifiers for the statements below. Case, apostrophes and wording are exactly those saved in Collection. The French glosses help us explain our reading of the expressions.

ID	Exact raw statement	French gloss
S01	J'ai n'est pas tchop depuis bayar	Je n'ai pas mangé depuis longtemps.
S02	je suis kass je go nang	Je suis fatigué, je vais dormir.
S03	je suis back du school le sharp man	Je suis revenu de l'école ce matin.
S04	j'ai mban gars je yabat	J'ai échoué, mon gars, j'ai mal.
S05	elles yamo flop le kongossa	Elles perdent leur temps à commérer.
S06	go au piol shoua mes kako	Va à la maison laver mes habits.

Reading the evidence

We read **tchop** in S01 as an eating verb because the gloss contains “mangé”. The current program emits NOUN, so we identify this as a questionable lexical label while keeping the computed table unchanged.

In S02, we read fatigue followed by an intention to sleep. The two clauses are placed next to each other. S03 combines a French frame with **back**, **school**, **sharp** and **man**. S04 expresses a setback and distress in its gloss. S05 concerns gossip, and S06 gives two successive household instructions.

These readings help us relate the category grammar to communication. The glosses can paraphrase the full expression rather than align with every word individually.

4 Raw Corpus: Statements S07–S12

We complete our fixed corpus with the six records below. We calculate the report totals from S01–S12 only, keeping public test history, dictionary entries and practice examples in their separate datasets.

ID	Exact raw statement	French gloss
S07	pere ca me chop je suis kass	Père, ça me touche, je suis épuisé.
S08	Ont a cote la light depuis le shap	On a coupé le courant depuis ce matin.
S09	shiba le price pere je n'ais pas flop les do	Baisse le prix, s'il te plaît, je n'ai pas beaucoup d'argent.
S10	mon combi came au school alli day a pied	Mon ami est venu à l'école, il est venu à pied.
S11	je falla le taco depuis bayar	Je cherche le taxi depuis longtemps.
S12	les talk free le francais	Ils parlent le français librement.

Interpretive themes

We read fatigue or personal state in S07, electricity interruption in S08, bargaining in S09, school and walking in S10, transport in S11, and language use in S12. Section 23 records the exact cues behind these interpretations.

We preserve `Ont a cote` in S08. In context, we read `a` as a French auxiliary, supported by the gloss “On a coupé le courant”. The unchanged lexer emits EF; our discussion makes that mismatch explicit. S09 retains `n'ais`, and S10 retains `alli`. Both remain unresolved and cause the first grammar failure in their respective statements.

Sampling boundary

Our twelve records meet the assignment’s numerical range. We use this small development corpus both to construct and to exercise the grammar; an independent sample would be needed to assess wider coverage.

5 Custom Lexical Specification

Segmentation by regular expression

We use a tokenizer that recognizes Unicode word-like runs, internal apostrophes/hyphens, decimal numbers, selected punctuation and finally any remaining non-whitespace character. The final alternative is important: unsupported input is preserved for classification rather than silently discarded.

The following factored notation describes the segmentation pattern. `M` denotes the combining-mark ranges `U+0300–036F`, `U+1AB0–1AFF`, `U+1DC0–1DFF`, `U+20D0–20FF` and `U+FE20–FE2F`. Substituting those ranges and `L` into the definitions yields the pattern in the source; spaces around the displayed alternatives are notation, not characters to match.

```
L      = [^\\W\\d_]+(?:[M]+[^\\W\\d_]*)*
WORD   = L(?:['\u2018\u2019\u02bc-]L)*
TOKEN = WORD | \\d+(?:[.,]\\d+)? | [.,!?:;"()] | \\S
```

Internal apostrophes therefore keep `J'ai`, `n'est` and `n'ais` as single tokens. Our twelve statements have no explicit punctuation tokens, but punctuation behavior is exercised by separate controls. Whitespace separates matches; the original full input, including its whitespace, is preserved independently of the token list.

Regex classification and fallback

```
NUMBER      ^\\d+(?:[.,]\\d+)?$
PUNCTUATION ^[.,!?:;"()]+$
ENGLISH_VERB_LIKE  suffix "ing" or "ed"
FRENCH_VERB_LIKE   suffix "er", "ir" or "re", length > 3
```

We apply the first two rules as full matches before lookup. We use the suffix checks on normalized words only after the explicit lexical sources have failed. We retain the `_LIKE` suffix to make a morphological guess visible; a verbal-looking ending can have a different function in context.

Deterministic pipeline

We tokenize the raw input, classify each token by the first applicable rule, retain its spelling and category, add phrase and language clues, compute statistics, and pass the categories to the parser. We keep the same input throughout these stages. The implementation is in `compiler/lexer/tokenizer.py` and `compiler/lexer/lexicon.py` [5].

6 Finite-State Model of the Lexer

We model the regular token pattern with an equivalent deterministic finite automaton (DFA). Figure 1 explains the pattern executed by Python’s `re` matcher. A double circle accepts a *token form*; vocabulary classification and full-statement parsing follow later.

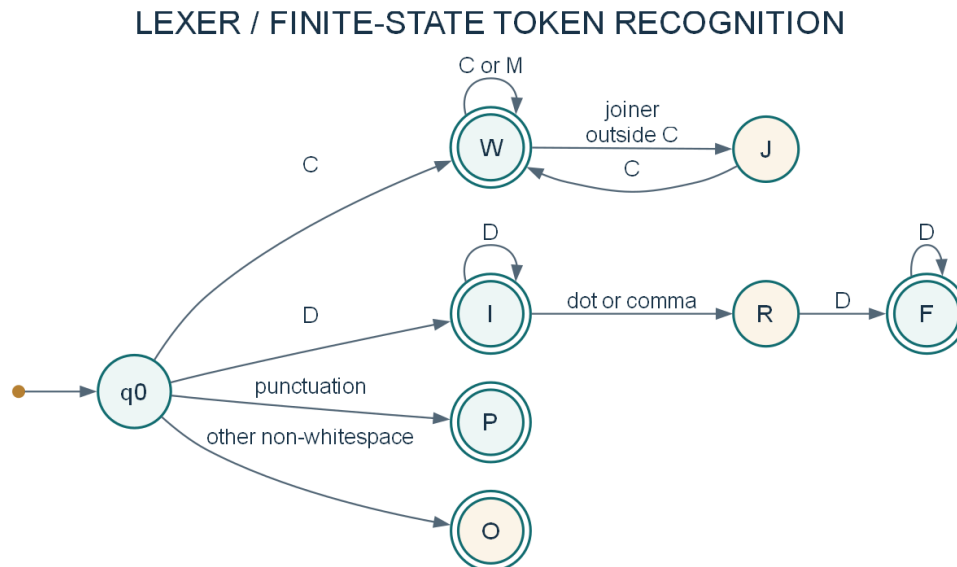


Figure 1. Equivalent token recognizer, initially in q_0 . Double-circle states accept: W word; I integer; F fraction; P punctuation; O other character. J awaits a joiner continuation and R a decimal digit; neither accepts. Unshown transitions enter an implicit dead state.

Input classes. C is `[^W\d_]`: a Unicode word character excluding decimal digits and underscore. D is `\d`; M is the combining-mark set on the preceding page. Pending joiners are ASCII/curly apostrophes or hyphen, outside C . `U+02BC` is already a word character and follows the C edges, even at a word boundary. Punctuation is `[.,!?:;"()]`; “other” is the remaining non-whitespace input, including a standalone underscore or hyphen.

Token boundaries. For this pattern, greedy matching retains the longest accepted prefix. A dangling joiner or decimal separator cannot finish in J or R , so the match ends at the previous accepting state and the remaining character is scanned again. Whitespace is skipped between matches, not emitted as a token. The source’s final `\S` alternative preserves other characters instead of losing them.

Raw input	Observed tokenization
<code>j'ai</code>	<code>["j'ai"]</code>
<code>go-slow</code>	<code>["go-slow"]</code>
<code>a-</code>	<code>["a", "-"]</code>
<code>3.14</code>	<code>["3.14"]</code>
<code>3.</code>	<code>["3", "."]</code>
<code>.5</code>	<code>[".", "5"]</code>
<code>-5</code>	<code>["-", "5"]</code>
<code>_x</code>	<code>["_", "x"]</code>

We executed these eight constructed checks through the real tokenizer. For example, `3.` becomes a number and punctuation, while `-5` becomes a hyphen and a number. A recognized word form can still receive `UNKNOWN` during classification.

7 Lexicon, Category Policy and Notation

Precedence is part of the result

We apply NUMBER/PUNCTUATION first, followed by compatible approved Word annotations. Conflicting reviewed labels are left unresolved. Our approved `choa` annotation occurs outside these twelve sentences.

Explicit lookup then proceeds through SLANG, PIDGIN_MARKER, NOUN, VERB, French function words and English function words. The reference CSV fills remaining gaps; one category is retained, multiple candidates yield AMBIGUOUS. Explicit French forms supplement gaps, then morphological guesses and finally UNKNOWN apply. We keep this precedence visible when comparing a software label with our reading of a word in context.

Our reference dictionary contains 938 display entries from 878 CSV rows and 60 additional senses. We use normalized aliases for lookup while keeping displayed spelling. Phrase annotations remain separate from the individual token categories.

Code	Lexer category	Code	Lexer category
FF	FRENCH_FUNCTION_WORD	EF	ENGLISH_FUNCTION_WORD
N	NOUN	V	VERB
ADJ	ADJECTIVE	ADV	ADVERB
AMB	AMBIGUOUS	PID	PIDGIN_MARKER
PUNC	PUNCTUATION	UNK	UNKNOWN
SL	SLANG	NUM	NUMBER

We abbreviate labels only in the printed tables. The lexer supports twenty categories; nine occur here. The grammar also uses nine terminals, replacing the corpus's UNK with optional PUNC.

Slang register and grammatical role

We identify the following 15 forms, occurring 17 times, as slang-register readings in our corpus. A slang word can also serve as a verb, noun or adjective. The application is unchanged: its emitted SLANG count remains zero, and all token tables below retain their actual labels.

Form(s)	Output	Our reading in context
nang, mban, yabat, yamo, shoua, falla	V	Verbal roles: sleeping, a setback, distress, attitude, washing and seeking.
piol, kako, taco, combi	N	Nominal roles: home, clothes, taxi and friend in the glosses.
kass	ADJ	Fatigue in S02 and S07 supports an adjective reading.
bayar	ADV	With <code>depuis</code> , it expresses a time span.
tchop	N	S01's eating meaning supports a verb reading.
kongossa	AMB	After <code>le</code> in S05, gossip is a nominal complement.
do	EF	S09's <code>les do</code> refers to money, a noun reading.

8 Token Tables: S01–S04

We show every emitted token in its original order. We use the preceding legend and preserve capitalization and apostrophes in the raw-token column.

S01 6 tokens
J'ai n'est pas tchop depuis bayar

No.	Raw token	Label
1	J'ai	V
2	n'est	V
3	pas	FF
4	tchop	N
5	depuis	FF
6	bayar	ADV

S02 6 tokens
je suis kass je go nang

No.	Raw token	Label
1	je	FF
2	suis	V
3	kass	ADJ
4	je	FF
5	go	V
6	nang	V

S03 8 tokens
je suis back du school le sharp man

No.	Raw token	Label
1	je	FF
2	suis	V
3	back	AMB
4	du	FF
5	school	N
6	le	FF
7	sharp	ADV
8	man	AMB

S04 5 tokens
j'ai mban gars je yabat

No.	Raw token	Label
1	j'ai	V
2	mban	V
3	gars	PID
4	je	FF
5	yabat	V

What these classifications show

We treat S01's J'ai and n'est as whole VERB tokens. Our grammar therefore works with coarser units than a detailed analysis that separates pronouns and auxiliaries.

We obtain two verbs from S02's go nang, enabling a serial-verb tail. S03 contains two ambiguous tokens without an UNKNOWN. In S04, we obtain PID for gars because the explicit Pidgin list has priority.

The lengths are six, six, eight and five tokens respectively. These 25 occurrences contribute to the corpus total once each.

9 Token Tables: S05–S08

We apply the same classification order to these four statements.

S05 5 tokens
elles yamo flop le kongossa

No.	Raw token	Label
1	elles	FF
2	yamo	V
3	flop	ADV
4	le	FF
5	kongossa	AMB

S06 6 tokens
go au piol shoua mes kako

No.	Raw token	Label
1	go	V
2	au	FF
3	piol	N
4	shoua	V
5	mes	FF
6	kako	N

S07 7 tokens
pere ca me chop je suis kass

No.	Raw token	Label
1	pere	N
2	ca	FF
3	me	AMB
4	chop	PID
5	je	FF
6	suis	V
7	kass	ADJ

S08 8 tokens
Ont a cote la light depuis le shap

No.	Raw token	Label
1	Ont	V
2	a	EF
3	cote	V
4	la	FF
5	light	N
6	depuis	FF
7	le	FF
8	shap	ADV

Interpretation

We obtain ADV for `flop` and AMB for `kongossa` in S05. The latter retains competing reference labels, although we read it as a nominal complement here. S06 places two instructions together. In S07, we obtain N for `pere`, FF for `ca` and PID for `chop`.

In S08, our linguistic reading of `a` is a French auxiliary. The software's English-word lookup still returns EF. We retain its V–EF–V sequence in the computed grammar evidence and identify the contextual auxiliary reading in this discussion.

These four statements contribute 26 occurrences. Along with the previous 25, they account for 51 of the 81 tokens.

10 Token Tables: S09–S12

We include both rejected originals in full. We complete lexical analysis before the parser stops at its first unsupported lookahead.

S09 10 tokens

shiba le price pere je n'ais pas flop les do

No.	Raw token	Label
1	shiba	V
2	le	FF
3	price	N
4	pere	N
5	je	FF
6	n'ais	UNK
7	pas	FF
8	flop	ADV
9	les	FF
10	do	EF

S10 9 tokens

mon combi came au school alli day a pied

No.	Raw token	Label
1	mon	FF
2	combi	N
3	came	V
4	au	FF
5	school	N
6	alli	UNK
7	day	N
8	a	EF
9	pied	N

S11 6 tokens

je falla le taco depuis bayar

No.	Raw token	Label
1	je	FF
2	falla	V
3	le	FF
4	taco	N
5	depuis	FF
6	bayar	ADV

S12 5 tokens

les talk free le francais

No.	Raw token	Label
1	les	FF
2	talk	V
3	free	AMB
4	le	FF
5	francais	N

The two unresolved forms

n'ais and alli each occur once and are the sixth token in their respective statement. They match no explicit reference, approved annotation or permitted morphological fallback. Each is therefore represented as UNK.

We read S09's do as money and S10's a pied as a walking expression with a French prepositional role. Both tokens retain EF in the unchanged program. These later labels are visible because we keep the complete lexical output even when parsing stops after five tokens.

The lengths here are ten, nine, six and five, contributing the final 30 occurrences.

11 Frequency, Coverage and Observed Variation

We count each of the twelve statements once. We case-fold forms for frequency counting and preserve their spelling in the token tables. Our sample contains 81 occurrences, 55 case-folded forms and a mean of 6.75 tokens per statement. Lengths range from five to ten.

				Case-folded form	Occurrences
Category	Code	Count	Share	je	7
FRENCH_FUNCTION_WORD	FF	28	34.6%	le	6
VERB	V	19	23.5%	depuis	3
NOUN	N	14	17.3%	suis	3
ADVERB	ADV	6	7.4%	j'ai	2
AMBIGUOUS	AMB	5	6.2%	pas	2
ENGLISH_FUNCTION_WORD	EF	3	3.7%	bayar	2
ADJECTIVE	ADJ	2	2.5%	kass	2
PIDGIN_MARKER	PID	2	2.5%	go	2
UNKNOWN	UNK	2	2.5%	school	2
				flop	2
				au	2
				pere	2
				a	2
				les	2

We obtain 47/81 occurrences in FF and V combined. The broad category definitions shape this distribution. We round each percentage independently.

Forty forms occur once

n'est, tchop, nang, back, du, sharp, man, mban, gars, yabat, elles, yamo, kongossa, piol, shoua, mes, kako, ca, me, chop, ont, cote, la, light, shap, shiba, price, n'ais, do, mon, combi, came, alli, day, pied, falla, taco, talk, free, francais.

Lookup coverage

We recognize 79 of 81 occurrences (97.5%), including ambiguous and context-insensitive labels. This is lookup coverage rather than tagging accuracy; we would need independent linguistic annotations to calculate precision and recall.

We use case-folding, Unicode decomposition, accent removal and curly-apostrophe normalization to propose groups. In this sample we obtain only J'ai and j'ai, one occurrence each. We discuss sharp/shap separately because their normalized strings differ.

12 Code-Mixing, Phrases and Linguistic Complexity

We study these expressions within Cameroon’s multilingual setting, where French, English, Pidgin and indigenous languages interact [3]. Ewondo and Fulfulde belong to that wider repertoire; our twelve transcriptions do not establish the individual speakers’ languages.

We obtain 26 candidate transition clues from the program’s FR, EN and PID word evidence. These clues describe lexical endpoints and leave meaning, intonation and conversational context to our interpretation.

ID	Exact emitted transition endpoints
S01	pas ... tchop; tchop ... depuis
S02	je ... go
S03	suis ... back; back ... du; du ... school; school ... le; le ... sharp; sharp ... man
S04	None emitted
S05	le ... kongossa
S06	go ... au
S07	ca ... chop; chop ... je
S08	la ... light; light ... depuis; le ... shap
S09	le ... price; price ... pere; les ... do
S10	mon ... combi; combi ... came; came ... au; au ... school; day ... pied
S11	None emitted
S12	les ... talk; free ... le

French frames and inserted words. In *je suis back du school*, we see a French pronoun, auxiliary and linking form around English-origin *back* and *school*. S07 also combines French material with *chop*, a Pidgin-associated form.

Meaning changes with local use. We read *flop* in S09 as “a lot” and *do* as money. Word-by-word English readings would miss the bargaining meaning of *pas flop les do*.

Spelling is variable. We retain *sharp* in S03 and *shap* in S08. Both glosses refer to the morning, suggesting a related reading that still needs contextual care. We keep both written forms, following the distinction between speech and transcription choices discussed by Ebongue [2].

Clauses can be juxtaposed. In *je suis kass je go nang*, we read fatigue followed by an intention to sleep. We understand the connection although the transcription contains neither a conjunction nor punctuation between the clauses.

Context and intonation complete the message. With *pere ca me chop*, tone and the preceding conversation would help us distinguish distress, complaint and emphasis. We therefore use the full gloss and surrounding words when explaining the expression. The fixed phrase regexes match zero verb or slang phrases in this corpus, even though these examples clearly contain colloquial constructions.

13 Constructing a Corpus-Derived Grammar

We build our CFG from recurring lexer-category structures. The original has ten nonterminals and 37 alternatives. For compact printing, we use Ut, Cl, St, Bt, Pt, Lp, Mn, At, Ct and Nm to abbreviate Utterance, Clause, SubjectTail, BareSubjectTail, PredicateTail, LinkedPhrase, ModifiedNominal, AdverbTail, ComplementTail and Nominal respectively.

LHS	Original alternatives
Ut	Cl Cl PUNC
Cl	FF St Nm Bt V Pt
St	V Pt Nm V Pt AMB PID Ct
Bt	V Pt FF St
Pt	V Pt EF V Pt ADJ Ct AMB Ct ADV At Nm Ct FF Lp PID Ct ϵ
Lp	V Pt Nm Ct AMB Ct ADV At FF Mn
Mn	Nm Ct AMB Ct ADV At
At	Nm Ct AMB Ct FF Lp V Pt ϵ
Ct	FF Lp V Pt ϵ
Nm	Nm N N

Evidence-to-rule rationale

We use Cl for a function-word subject, a nominal opening or a verb-initial clause. We use St for *je suis*, *mon combi came* and the observed AMB–PID pattern in *ca me chop*. Pt permits auxiliary/serial verbs (*j'ai mban*, *go nang*), observed V–EF–V links, and nominal, adjectival, adverbial or ambiguous complements.

Lp and Mn represent introduced groups such as *au piol*, *depuis bayar* and *depuis le shap*. Ct allows juxtaposition, as in *je suis kass je go nang* and *go au piol shoua mes kako*. Nm represents noun clusters; its natural left recursion is intentionally processed by the real transformation routine.

We let Ut end with one optional punctuation token and test this extension separately. We exclude UNKNOWN and allow AMB only in explicit positions. The calculations here use the corpus grammar recorded with our evidence, including its actual V–EF–V sequence for S08.

14 Grammar Transformations

Step 1: eliminate direct left recursion

We detect direct left recursion in the original nonterminal N_m :

$$N_m \rightarrow N_m N \mid N.$$

We introduce `Nominal_LR1`, abbreviated N_r , and obtain:

$$N_m \rightarrow N N_r, \quad N_r \rightarrow N N_r \mid \epsilon.$$

Both forms generate one or more noun-category tokens. The original extends a noun chain by left recursion; the transformed form consumes the first noun and extends it through a right-recursive suffix. No raw noun is deleted and the language of this subgrammar is unchanged.

Step 2: left-factor the start rule

The original start rule has a shared prefix:

$$U_t \rightarrow C_l \mid C_l \text{ PUNC}.$$

We introduce `Utterance_LF1`, abbreviated U_f :

$$U_t \rightarrow C_l U_f, \quad U_f \rightarrow \epsilon \mid \text{PUNC}.$$

The alternative choice is postponed until the clause has been processed. End-of-input selects epsilon; PUNC selects the optional punctuation. The empty alternative is a suffix, not permission for an empty whole utterance.

Our transformation record

We record the two operations as `eliminate_direct_left_recursion` and `left_factor`. Each record contains the complete grammar before and after the operation. This rule ordering requires no indirect-substitution step. We also check for residual nullable-prefix recursion.

We obtain twelve nonterminals, 39 alternatives, no residual left recursion and no LL(1) conflict. Our regression checks compare the original and transformed languages with an independent derivation oracle up to four terminals.

15 Transformed Production Catalogue

We assign P01–P39 in the computed grammar’s order and use them consistently in the predictive table and traces. Nr is `Nominal_LR1`; Uf is `Utterance_LF1`.

ID	Transformed production	ID	Transformed production
P01	Ut $\rightarrow$ Cl Uf	P21	Lp $\rightarrow$ AMB Ct
P02	Cl $\rightarrow$ FF St	P22	Lp $\rightarrow$ ADV At
P03	Cl $\rightarrow$ Nm Bt	P23	Lp $\rightarrow$ FF Mn
P04	Cl $\rightarrow$ V Pt	P24	Mn $\rightarrow$ Nm Ct
P05	St $\rightarrow$ V Pt	P25	Mn $\rightarrow$ AMB Ct
P06	St $\rightarrow$ Nm V Pt	P26	Mn $\rightarrow$ ADV At
P07	St $\rightarrow$ AMB PID Ct	P27	At $\rightarrow$ Nm Ct
P08	Bt $\rightarrow$ V Pt	P28	At $\rightarrow$ AMB Ct
P09	Bt $\rightarrow$ FF St	P29	At $\rightarrow$ FF Lp
P10	Pt $\rightarrow$ V Pt	P30	At $\rightarrow$ V Pt
P11	Pt $\rightarrow$ EF V Pt	P31	At $\rightarrow$ ϵ
P12	Pt $\rightarrow$ ADJ Ct	P32	Ct $\rightarrow$ FF Lp
P13	Pt $\rightarrow$ AMB Ct	P33	Ct $\rightarrow$ V Pt
P14	Pt $\rightarrow$ ADV At	P34	Ct $\rightarrow$ ϵ
P15	Pt $\rightarrow$ Nm Ct	P35	Nm $\rightarrow$ N Nr
P16	Pt $\rightarrow$ FF Lp	P36	Nr $\rightarrow$ N Nr
P17	Pt $\rightarrow$ PID Ct	P37	Nr $\rightarrow$ ϵ
P18	Pt $\rightarrow$ ϵ	P38	Uf $\rightarrow$ ϵ
P19	Lp $\rightarrow$ V Pt	P39	Uf $\rightarrow$ PUNC
P20	Lp $\rightarrow$ Nm Ct		

How to use the catalogue

P01 starts every parse by expanding Ut into Cl and Uf. P02–P04 select a clause opening from FF, N or V. S02 then selects P05 and P12 for its verb and adjective complement. Later P32 and P19 admit the juxtaposed phrase, and P10 admits its next verb. The empty suffixes P18 and P38 complete the parse.

P35–P37 describe noun clusters. P37 can be chosen only when the next token belongs to FOLLOW(Nr), not for every unexpected token. P38 and P39 preserve the original optional single-punctuation choice. The table on the next two pages makes each choice explicit rather than relying on procedural “try alternatives until one works” logic.

We retain full names, alternatives, generated helpers and transformation records in the companion JSON. The abbreviations keep the printed calculations readable.

16 FIRST and FOLLOW Sets

We calculate FIRST and FOLLOW for every transformed nonterminal. FIRST contains possible initial terminals of a derivation and epsilon when the nonterminal is nullable. FOLLOW contains terminals, or the end marker \$, that can immediately follow a nonterminal in a derivation from the start symbol. Epsilon does not belong in FOLLOW.

NT	FIRST	FOLLOW
Ut	FF, N, V	\$
Cl	FF, N, V	\$, PUNC
St	AMB, N, V	\$, PUNC
Bt	FF, V	\$, PUNC
Pt	ADJ, ADV, AMB, EF, FF, N, PID, V, ϵ	\$, PUNC
Lp	ADV, AMB, FF, N, V	\$, PUNC
Mn	ADV, AMB, N	\$, PUNC
At	AMB, FF, N, V, ϵ	\$, PUNC
Ct	FF, V, ϵ	\$, PUNC
Nm	N	\$, FF, PUNC, V
Nr	N, ϵ	\$, FF, PUNC, V
Uf	PUNC, ϵ	\$

Fixed-point computation

For each production, FIRST accumulates the first symbol's terminal evidence, continuing through nullable prefixes and adding epsilon only when the entire right-hand side is nullable. The implementation repeats these updates until no set grows. FOLLOW starts by adding \$ to Ut. For each nonterminal within a right-hand side, FIRST of the suffix contributes non-epsilon terminals; a nullable or absent suffix also contributes FOLLOW of the left side. Those updates likewise continue to a fixed point.

Sanity checks on this grammar

FIRST(Ut) is exactly {FF, N, V}; hence empty input cannot begin a clause. FOLLOW(Ut) contains only end-of-input. Uf is nullable and permits punctuation, which explains PUNC in FOLLOW(Cl). Nr can begin another N or become empty, but its FOLLOW set is {\$, FF, PUNC, V}, not UNKNOWN.

We generate these sets directly from the transformed productions. The full sets let us check epsilon entries and explain the rejection at the sixth token in S09 and S10.

17 Complete LL(1) Predictive Table

For $A \rightarrow \alpha$, each terminal in $\text{FIRST}(\alpha) \setminus \{\epsilon\}$ receives that production. If α is nullable, its production is also placed under every terminal in $\text{FOLLOW}(A)$. Multiple distinct productions in one cell are conflicts; the implementation retains the conflict and does not choose an arbitrary winner.

We print the complete table below. Columns are the nine grammar terminals plus end-of-input. The 47 populated cells refer to P01–P39 in the catalogue. “–” means no production and causes rejection for that nonterminal/lookahead.

NT	ADJ	ADV	AMB	EF	FF	N	PID	PUNC	V	\$
Ut	–	–	–	–	P01	P01	–	–	P01	–
Cl	–	–	–	–	P02	P03	–	–	P04	–
St	–	–	P07	–	–	P06	–	–	P05	–
Bt	–	–	–	–	P09	–	–	–	P08	–
Pt	P12	P14	P13	P11	P16	P15	P17	P18	P10	P18
Lp	–	P22	P21	–	P23	P20	–	–	P19	–
Mn	–	P26	P25	–	–	P24	–	–	–	–
At	–	–	P28	–	P29	P27	–	P31	P30	P31
Ct	–	–	–	–	P32	–	–	P34	P33	P34
Nm	–	–	–	–	–	P35	–	–	–	–
Nr	–	–	–	–	P37	P36	–	P37	P37	P37
Uf	–	–	–	–	–	–	–	P39	–	P38

Worked entries

- $M[\text{Ut}, \text{FF}] = \text{P01}$, since Cl can begin with FF and Uf follows it. The same production is selected under N and V.
- $M[\text{Pt}, \text{ADJ}] = \text{P12}$, selecting $\text{Pt} \rightarrow \text{ADJ Ct}$.
- $M[\text{Pt}, \$] = \text{P18}$, since the Pt alternative is empty and end-of-input is in $\text{FOLLOW}(\text{Pt})$.
- $M[\text{Nr}, \text{N}] = \text{P36}$, whereas FF, V, PUNC and \$ select its empty alternative P37.
- Uf selects P38 at \$ and P39 at PUNC. No row has a defined UNKNOWN entry.

LL(1) conclusion

We find no competing production in a cell and no residual left recursion, so predictive parsing is enabled. Our category grammar covers a limited subset of everyday expressions.

18 Pushdown Automaton for the LL(1) Parser

We model our bounded parser as a deterministic pushdown recognizer. Its extra memory is the symbol stack. For our grammar, the stack alphabet contains twelve nonterminals, nine terminals and $\$$. We feed it lexer categories and append a virtual $\$$ to mark the end.

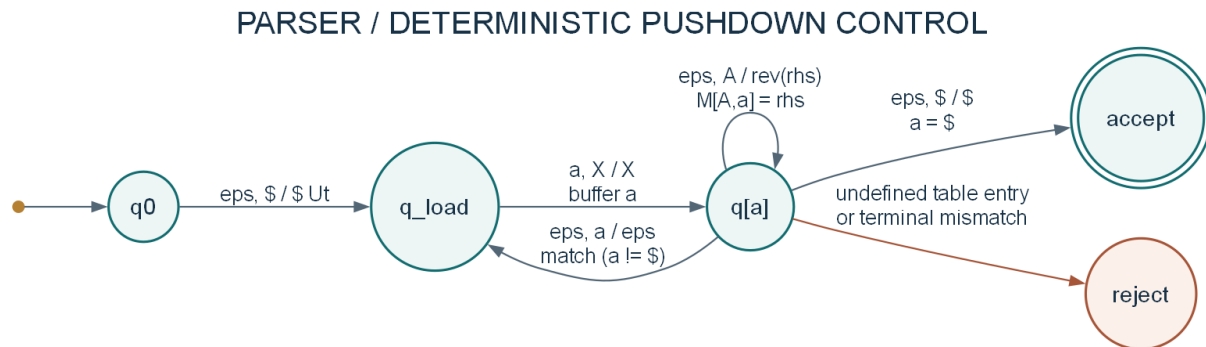


Figure 2. Finite control of the predictive pushdown automaton. An edge label is “input, stack top / replacement”; eps consumes no input. The stack top is at the right. $q[a]$ abbreviates one control state per possible buffered lookahead, not an unbounded variable.

Expand, match, accept. The initial stack becomes $[\$, Ut]$. With a nonterminal A on top, the unique table entry $M[A, a]$ replaces it by the reversed right-hand side without consuming the buffered category. An epsilon production only pops A . If a terminal matches a , it is popped and the next lookahead is loaded. Acceptance requires $\$$ on top and at input end; the implementation’s final trace still shows the bottom marker rather than an empty stack.

Why this is deterministic. Loading reads one category into finite control; expansion and matching then use epsilon transitions. The table has no conflicting entry, so each valid configuration selects at most one action. There are twenty supported lexer categories plus the end marker. The diagram uses a prefetched lookahead; the actual code instead reads `categories[position]` and advances `position` only on a match. This equivalent presentation does not change the accepted category sequences.

Concrete execution. In S02, the first expansion changes $[\$, Ut]$ to $[\$, Uf, Cl]$. Later matching FF consumes the first token. S09/S10 instead reach an undefined table entry with UNK after five matches, so neither the unknown nor the following tokens is skipped. The complete traces on the next pages show the recorded configurations.

Implemented guard	Bound
Input tokens	256
Characters in an individual token	1,000
Recorded parse steps	2,000
Stack symbols	1,024

Malformed tokens and non-LL(1) grammars are rejected before the stack loop. The implementation also checks repeated configurations and the bounds above. Trace snapshots precede each action.

19 Complete Accepted Trace: S02

We trace `je suis kass je go nang`, whose emitted categories are FF, V, ADJ, FF, V, V. The table contains every recorded step; production identifiers refer to the full catalogue. Each stack/remaining pair precedes its action.

Step	Stack (top at right)	Remaining input	Action
1	\$ Ut	FF V ADJ FF V V \$	P01
2	\$ Uf Cl	FF V ADJ FF V V \$	P02
3	\$ Uf St FF	FF V ADJ FF V V \$	match FF
4	\$ Uf St	V ADJ FF V V \$	P05
5	\$ Uf Pt V	V ADJ FF V V \$	match V
6	\$ Uf Pt	ADJ FF V V \$	P12
7	\$ Uf Ct ADJ	ADJ FF V V \$	match ADJ
8	\$ Uf Ct	FF V V \$	P32
9	\$ Uf Lp FF	FF V V \$	match FF
10	\$ Uf Lp	V V \$	P19
11	\$ Uf Pt V	V V \$	match V
12	\$ Uf Pt	V \$	P10
13	\$ Uf Pt V	V \$	match V
14	\$ Uf Pt	\$	P18
15	\$ Uf	\$	P38
16	\$	\$	accept

Explanation of the derivation

P02 chooses the FF-led clause. After matching `je`, P05 provides a verb and predicate tail. P12 selects an adjective complement for `kass`. P32 consumes the next FF and P19 allows a following verb, modeling the juxtaposition rather than insisting on punctuation or a conjunction.

P10 permits the second verb in `go nang`. Once all six input categories have been consumed, P18 removes the empty predicate tail and P38 removes the optional punctuation suffix. The final row sees only the two end markers and accepts. The consumed count is six and the error field is null.

What acceptance means

We accept this category pattern after consuming all six tokens. Vocabulary approval also passes because every token is recognized. Our constructed known-word controls show why we calculate the two verdicts separately.

20 Complete Rejected Traces: S09 and S10

We complete lexical analysis of both inputs, then reject at the first unavailable parsing-table entry. We preserve the unknown token in the trace.

S09: n'ais at position 6.

Step	Stack (top at right)	Remaining input	Action
1	\$ Ut	V FF N N FF UNK FF ADV FF EF \$	P01
2	\$ Uf Cl	V FF N N FF UNK FF ADV FF EF \$	P04
3	\$ Uf Pt V	V FF N N FF UNK FF ADV FF EF \$	match V
4	\$ Uf Pt	FF N N FF UNK FF ADV FF EF \$	P16
5	\$ Uf Lp FF	FF N N FF UNK FF ADV FF EF \$	match FF
6	\$ Uf Lp	N N FF UNK FF ADV FF EF \$	P20
7	\$ Uf Ct Nm	N N FF UNK FF ADV FF EF \$	P35
8	\$ Uf Ct Nr N	N N FF UNK FF ADV FF EF \$	match N
9	\$ Uf Ct Nr	N FF UNK FF ADV FF EF \$	P36
10	\$ Uf Ct Nr N	N FF UNK FF ADV FF EF \$	match N
11	\$ Uf Ct Nr	FF UNK FF ADV FF EF \$	P37
12	\$ Uf Ct	FF UNK FF ADV FF EF \$	P32
13	\$ Uf Lp FF	FF UNK FF ADV FF EF \$	match FF
14	\$ Uf Lp	UNK FF ADV FF EF \$	reject

S10: alli at position 6.

Step	Stack (top at right)	Remaining input	Action
1	\$ Ut	FF N V FF N UNK N EF N \$	P01
2	\$ Uf Cl	FF N V FF N UNK N EF N \$	P02
3	\$ Uf St FF	FF N V FF N UNK N EF N \$	match FF
4	\$ Uf St	N V FF N UNK N EF N \$	P06
5	\$ Uf Pt V Nm	N V FF N UNK N EF N \$	P35
6	\$ Uf Pt V Nr N	N V FF N UNK N EF N \$	match N
7	\$ Uf Pt V Nr	V FF N UNK N EF N \$	P37
8	\$ Uf Pt V	V FF N UNK N EF N \$	match V
9	\$ Uf Pt	FF N UNK N EF N \$	P16
10	\$ Uf Lp FF	FF N UNK N EF N \$	match FF
11	\$ Uf Lp	N UNK N EF N \$	P20
12	\$ Uf Ct Nm	N UNK N EF N \$	P35
13	\$ Uf Ct Nr N	N UNK N EF N \$	match N
14	\$ Uf Ct Nr	UNK N EF N \$	reject

S09 ends at Lp (LinkedPhrase) with lookahead UNK; its legal lookaheads are ADV, AMB, FF, N and V. S10 ends at Nr (Nominal_LR1) with UNK; legal lookaheads are \$, FF, N, PUNC and V. Both have consumed five tokens. The different stack tops explain the distinct error messages even though the failure positions match.

21 Evaluation Results and Boundary Controls

We evaluate all twelve originals with the same grammar and reviewed lexicon snapshot. We compute every token, verdict, consumed count and trace length shown below.

ID	Tokens	Vocabulary	CFG	Consumed	Trace rows
S01	6	Pass	Accept	6	18
S02	6	Pass	Accept	6	16
S03	8	Pass	Accept	8	22
S04	5	Pass	Accept	5	14
S05	5	Pass	Accept	5	14
S06	6	Pass	Accept	6	20
S07	7	Pass	Accept	7	19
S08	8	Pass	Accept	8	21
S09	10	Fail	Reject	5	14
S10	9	Fail	Reject	5	14
S11	6	Pass	Accept	6	18
S12	5	Pass	Accept	5	16

We accept ten category sequences (83.3%) and reject two (16.7%). We then use the following constructed controls to check the distinction between vocabulary approval and grammar acceptance.

ID	Constructed control	Vocabulary	CFG
C01	(empty)	Pass	Reject
C02	school	Pass	Reject
C03	je	Pass	Reject
C04	je suis kass.	Pass	Accept
C05	je suis kass?!	Pass	Reject
C06	je je suis	Pass	Reject
C07	je suis alli	Fail	Reject

Empty input has no UNKNOWN but cannot derive a Clause. A noun-only fragment and a dangling FF-led clause also reject. One punctuation token is supported; two are not. Known categories in an unsupported order can reject without any unknown word.

We also test unseen combinations, unsupported category orders and bounded language equivalence. A larger independently annotated corpus would help us assess how well the grammar generalizes.

22 Working Analyzer: Browser Evidence

We illustrate the workflow with captures from <https://camfranglais.duckdns.org> dated 27 September 2026. We inspected an existing saved test and its original results.

SELECTED SAVED TEST

Analyzed sentence or word

Recorded 9/25/2026, 7:49:11 AM. These are the original results, not a new analysis using today's settings.

Publicly retained original text and results. Saved tests are immutable.

je suis kass je go nang

ACCEPT

No UNKNOWN tokens. Recognized slang and all other known lexer categories are accepted.

Vocabulary approval is not grammatical correctness. The CFG grammar check is shown separately in Analysis.

Use as analyzer input →

▼ Token details for this test

#	Observed text	Lexer category
1	je	FRENCH_FUNCTION_WORD
2	suis	VERB
3	kass	ADJECTIVE
4	je	FRENCH_FUNCTION_WORD
5	go	VERB
6	nang	VERB

Screenshot. Existing S02 selected through Analysis, with its original input, vocabulary verdict and token details.

Franc Analyzer

Split into tokens, classify using the shared vocabulary, then test the saved grammar.

Statement to analyze

Type or paste a sentence or word. Leave blank to test empty input (epsilon).

0 / 4,000 characters · not saved to Collection

<> Analyze

Using saved grammar

Analyze publicly retains this test's original text, saved grammar and results, even when rejected. Do not submit personal or confidential content. Repeated completed tests count separately. Nothing is added to Collection.

Camfranglais - Compiler Workspace

Data & privacy →

Screenshot. The current analyzer opens directly without login. Submission explicitly warns that tests are public and retained. This crop shows the input panel.

We retrieve S02's original token table and trace when selecting its saved test. The capture register and uncropped images accompany the report.

23 Topic Patterns in the Corpus

We relate each statement to an everyday theme using its wording and French gloss. The table gives the exact cues behind our reading, making it clear how we arrived at each topic. These are our interpretations of the text, rather than an additional output of the application.

ID	Textual cues	Our topic reading
S01	tchop	Food and eating
S02	kass, nang	Fatigue and rest
S03	back, school	School and a return journey
S04	mban, yabat	Setback and personal distress
S05	kongossa	Gossip and social talk
S06	piol, shoua, kako	Household tasks
S07	kass	Fatigue and personal state
S08	cote, light	Electricity interruption
S09	shiba, price, do	Market bargaining and transactions
S10	came, school, a pied	Walking to school; commuting
S11	falla, taco	Seeking transport
S12	talk, francais	Language use

How we read the patterns

We pair `cote` with `light` for electricity and `shiba`, `price` and `do` for bargaining. `school` suggests university or school life; movement words and `a pied` add the commuting reading. We read `taco` as transport because the corresponding gloss explicitly mentions a taxi.

We use surrounding words to handle ambiguous cues. In S02, `go` occurs with sleeping and fatigue, so we prefer rest over a transport label. Security and weather remain unrepresented in these twelve statements. Our analysis covers the observed themes without adding new raw statements.

Reference material

We consult Collection for the original entries and Dictionary for source-labelled meanings. We keep the 26 synthetic practice examples separate from this corpus. The two screenshots in Section 22 illustrate the implemented analyzer, while the table above presents our topic readings.

24 Application Architecture

We organize the application into four layers. The interface receives a statement, the backend coordinates the request, the compiler produces the lexical and grammar results, and storage preserves the test. We describe the detailed classes and interactions in the separate SDD and UML atlas.

Layer	Responsibility
Frontend	React/TypeScript pages for input, results, Collection, Dictionary and synthetic examples.
Backend	FastAPI request handling, analysis coordination and retrieval of saved results.
Lexer / parser	Regex and vocabulary rules, frequencies, grammar transformations, FIRST/FOLLOW, the LL(1) table and parsing traces.
Storage	SQLite records for Collection, the saved grammar and original test snapshots.

25 Verification, Reproducibility and Limitations

Our checks

We compare the report with a fresh computation from the unchanged twelve statements. This revision improves the writing and linguistic discussion while preserving the application, grammar and measured compiler results.

Evidence	Observed result
Raw corpus	All twelve strings match their fixtures exactly.
Lexical output	All 81 tokens, frequencies, category counts and variations match the recorded evidence.
Grammar	The two transformations yield 39 productions, 47 populated LL(1) cells and zero conflicts.
Parser	Ten originals accept; S09 and S10 reject at token six. The complete traces and constructed controls match their expected results.
Report	Generated tables, embedded figures, references and PDF layout are checked against the source material.

Reproduction procedure

We run `python -m tools.build_report_evidence -check`, the corpus regression tests, the local LaTeX build and `python -m tools.check_documentation`. The JSON retains full grammar names, intermediate transformations and traces.

Our main limits are the small development corpus, broad lexical categories and unresolved forms. The linguistic readings in Sections 7 and 23 help us explain those limits without replacing the measured outputs.

26 Conclusion

We have built and examined a complete compiler pipeline around twelve unchanged transcriptions. We classify all 81 token occurrences, compute frequencies and language clues, transform the grammar and expose the conflict-free LL(1) table. The traces show how each lexical category affects the next parsing action.

We accept ten statements and locate the unresolved forms in S09 and S10 at token six. Our controls also show that recognized words can occur in an unsupported category order. Through the slang and topic readings, we relate the formal output to everyday meaning, variable spelling and juxtaposed clauses.

We present these stages in a public interface that keeps original test results available for inspection. The project has helped us connect regex recognition, grammar transformations and predictive parsing with the complexity of informal communication.

Submission and presentation

We identify `tchop`, the contextual uses of `a`, and the two unknown forms as priorities for later lexical review. We preserve the current compiler results so that the report's tables and traces remain reproducible.

Our deadline is **29 September 2026 at 14:00**. In the examination, **each student presents for three minutes**. We will rehearse our individual parts and use an existing test to explain the main workflow.

Author group

We present this work jointly as the three-member group below. We share credit for entering the twelve statements into the compiler and preparing the corpus used in this report.

Member	Matricule
Kwete Ngouba Junior Rayan	ICTU20241377
Djemtchimo Noukui Bruno Jonatan	ICTU20241585
Amina Boubakary	ICTU20241870

References and Evidence Register

- [1] Faculty of Information and Communication Technologies. *CS4110 Compiler Construction, Summer 2026, SET A: Lexical and Syntactic Analysis of Informal Urban Communication in Yaoundé*. Assignment brief; instructor Engr. Tanwi Nkiambo; assessment dated 29 September 2026. Source of the 10–15-statement, group-of-three and 25–30-page requirements.
- [2] Augustin Emmanuel Ebongue. “Linguistic Features in a Marginal Corpus. The Case of Written Camfranglais, a Cameroon Youth Language.” *Multilingual Margins*, 9(1), issue year 2022; published online 25 May 2023. Publisher metadata and abstract checked 27 September 2026. <https://www.epubs.ac.za/index.php/mm/article/view/1392>.
- [3] Valentine Njende Ubanako. “Is ‘Camfranglais’ A New Language? A Review of Current Opinions.” *International Linguistics Research*, 4(1), 2021. We use its discussion of Cameroon’s wider language-contact setting. <https://doi.org/10.30560/ilr.v4n1p36>.
- [4] Alfred V. Aho, Monica S. Lam, Ravi Sethi and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*, second edition. Pearson/Addison-Wesley. General compiler background for lexical analysis, grammar transformations and predictive parsing.
- [5] Camfranglais project source, baseline `a1b88aa.compiler/lexer/tokenizer.py`, `compiler/lexer/languages.py`, `compiler/lexer/reference.py`, `compiler/parser/yaounde.py`, `compiler/parser/analysis.py`, `compiler/parser/predictive.py`. We reproduce the reported outputs from these implemented rules.
- [6] Project Collection, twelve exact transcriptions `S01–S12.docs/evidence/final-corpus-20260927.json`. Read-only analysis snapshot dated 27 September 2026, preserving raw wording and French glosses. The initial compiler-input record is dated 22 September 2026; Section 2 describes our corpus preparation.
- [7] Reproducible coursework analysis. `tools/build_report_evidence.py`, `docs/evidence/final-analysis-20260927.json`, `docs/report-data/`. Contains source fingerprints, full tokens, sets, table, transformations, traces and boundary controls. Tested against the twelve exact cases in `compiler/tests/yaounde_cases.py`.
- [8] Classified vocabulary and additional reference senses. `dictionary/full_lexicon_classified.csv`, `dictionary/camfranglais.md`, `dictionary/extra_lexicon.md`. Source-labelled lookup material: 878 CSV rows and 938 dictionary display entries.
- [9] Public application screenshots, captured 27 September 2026. `docs/evidence/screenshots/`; <https://camfranglais.duckdns.org>. We inspected existing records without posting new tests. The capture register records image hashes, context and the saved-test crop.

Supplementary SRS, SDD and UML atlas sources and PDFs are distributed in the same documentation directory.